FlowDepthFormer: Lightweight Video Stabilization with Temporal Transformers

Bhavisha Chaudhari, Adarsh Jha, Mitul Nagar
Department of Computational Intelligence
SRM Institute of Science and Technology, Kattankulathur
Chennai, India
{bc2780, aj4391, mituln}@srmist.edu.in

***Abstract*—We present a new real-time video stabilization method that adds 3D understanding by combining monocular depth estimation with dense optical flow and temporal transformers. Unlike traditional 2D methods that move frames evenly, our approach creates pseudo-3D motion fields that respect the scene's depth and parallax effects. This lets us warp frames more accurately, improving overall visual quality and stability. To keep the video smooth over time, we use a small transformer to reduce jitter in the camera path, which works better than classic smoothing filters like Gaussian blur. Our method does not need training and runs fast, achieving over 30 frames per second on 720p videos. We tested it on videos from handheld cameras, drones, and wearable devices, showing it gives better stability and keeps the scene's structure well. This work blends older stabilization techniques with new 3D scene tools, making it a strong, fast option for phone videos, drone shots, and AR/VR applications.**

***Index Terms*—Video stabilization, depth-flow fusion, optical flow, temporal smoothing, real-time processing**

I. Introduction

Video stabilization is essential for improving the visual quality of footage captured by handheld devices, drones, and wearable cameras. Unintended camera motion such as shake or jitter can degrade user experience and hinder downstream tasks like object detection, tracking, or augmented reality overlays [1], [2]. While hardware-based solutions such as gimbals or optical image stabilization (OIS) offer effective motion compensation, they are often bulky, expensive, and unsuitable for mobile or embedded platforms [3].

As a result, software-based stabilization techniques have gained popularity due to their flexibility and cost-effectiveness. Classical approaches typically estimate camera motion using feature tracking or optical flow and then apply global transforms to smooth trajectories [4], [5]. However, these 2D methods often fall short in scenes with significant depth variation or parallax. Applying uniform transformations across the entire frame can introduce visible warping and geometric distortions, especially when foreground and background objects move differently [6], [7].

To overcome these limitations, we propose a 3D-aware video stabilization framework that combines monocular depth estimation with dense optical flow to enable context-aware motion compensation. We use the MiDaS model [8] to extract relative depth from single RGB frames and fuse it with motion vectors derived from Farneback optical flow [9]. This fusion helps infer pseudo-3D motion fields, allowing our system to preserve geometric consistency across varying scene depths.

To further improve temporal coherence, we introduce a lightweight transformer module that adaptively smooths motion trajectories. Unlike fixed filters such as Gaussian or moving averages [11], our transformer dynamically learns motion trends to reduce high-frequency jitter while preserving intended movement. The transformer operates without pre-training, maintaining the modularity and efficiency of the pipeline.

Our framework is designed for real-time deployment and achieves over 30 FPS on 720p videos using consumer-grade GPUs. It is training-free, hardware-efficient, and suitable for use in mobile devices, drone capture, and AR/VR systems.

Extensive experiments across handheld, aerial, and egocentric videos show that our method consistently outperforms prior works such as DeepStab [4] and StabNet [12] in terms of stability, distortion, and frame retention. By combining classical motion estimation with modern 3D-aware cues, our approach offers a robust and scalable solution for real-time video stabilization.

II. Related Work

A. Classical Video Stabilization

Traditional video stabilization methods are broadly categorized into feature-based and direct (pixel-based) approaches. Feature-based techniques detect and track keypoints across frames using descriptors like SIFT [13], SURF [14], ORB [15], or KLT [16]. These tracked features are used to estimate camera motion, typically modeled using affine or projective transformations [1]. While effective in many scenarios, such methods often fail in textureless regions, under drastic lighting changes, or in the presence of motion blur.

Direct methods, on the other hand, rely on intensity differences across pixels, often using optical flow to compute motion fields. Algorithms such as Lucas-Kanade [17] and Farneback [9] are widely used for this purpose. These methods offer dense motion estimation and are robust in low-feature environments. However, classical stabilizers applying uniform warping based on these motion estimates often produce artifacts in scenes with depth variation or parallax [18].

Trajectory smoothing is another key component in classical methods. Common techniques include moving average,

Kalman filters, and Gaussian smoothing kernels [11], [19]. While computationally efficient, these methods use fixed assumptions about motion patterns and often underperform in highly dynamic scenarios.

B. Learning-Based Stabilization

Recent advancements in deep learning have led to the development of data-driven video stabilization models. DeepStab [4] uses a convolutional neural network trained on synthetic jittered videos to predict frame-to-frame stabilization transforms. StabNet [12] employs an encoder-decoder architecture to estimate warping grids directly from input sequences. These models demonstrate strong performance in many cases but require large annotated datasets and suffer from poor generalization to unseen content or camera types.

To improve generalization, DEFNet [20] introduces deformable convolution layers, and NVDS+ [21] incorporates semantic segmentation and depth cues. However, these models often come with increased complexity, latency, and memory usage, making them less suitable for real-time or mobile applications.

Other works like Deep Online Video Stabilization (DOVS) [22] attempt real-time learning but rely on pretraining with long video sequences. These methods are also limited in their ability to handle high-parallax scenes or strong 3D effects, as they primarily operate in the 2D image plane.

C. Depth-Aware and 3D-Aware Stabilization

Incorporating depth cues has shown promise in improving stabilization performance. Kopf et al. [23] created a 3D mesh of the scene using structure-from-motion to stabilize hyperlapse videos, but their method is resource-intensive and unsuitable for real-time use. Other methods [24], [25] use planar segmentation and layered motion estimation to address parallax, but they rely on external calibration or precomputed scene geometry.

With the advent of monocular depth estimation, methods like MiDaS [8], Monodepth2 [26], and DPT [27] enable lightweight depth prediction from single RGB frames. MiDaS, in particular, is known for its strong generalization due to its multi-dataset training regime. Some works have explored using this depth for stabilization [22], [28], but the integration is often shallow or used only for post-processing.

Our work builds upon this line by fusing dense optical flow and monocular depth into a unified 3D motion field, allowing more accurate and geometry-aware frame warping in dynamic scenes.

D. Transformers in Video and Motion Modeling

Transformer-based architectures have recently been applied to various video tasks such as object tracking, motion prediction, and video understanding [29], [30]. Their ability to model long-range dependencies makes them well-suited for temporal smoothing of camera trajectories. Some lightweight variants, like the MobileViT [31] and TinyViT [32], have shown effectiveness on edge devices.

To our knowledge, no prior work combines temporal transformers with depth-flow fusion for real-time video stabilization. Our approach introduces a lightweight transformer module that adaptively smooths trajectories without training on stabilization datasets, improving robustness and generalization across video types.

III. Methodology

A. System Overview

Our proposed 3D-aware video stabilization framework is designed to integrate dense optical flow, monocular depth estimation, and temporal smoothing using transformers. The architecture follows a modular pipeline that processes raw video frames and outputs geometrically consistent, stabilized video.

Given an input video stream, each frame undergoes preprocessing (resizing, normalization, and grayscale conversion) to prepare for subsequent analysis. The pipeline then branches into two parallel modules: one computes dense motion fields via the Farneback optical flow algorithm, while the other infers scene geometry using a monocular depth estimator such as MiDaS.

The outputs of both modules are fused in the depth-flow fusion block to generate pseudo-3D motion vectors, which more accurately reflect scene dynamics, especially in environments with depth discontinuities or parallax. These vectors are aggregated over time to compute a raw camera trajectory.

To reduce high-frequency jitter and preserve intentional movement, the trajectory is smoothed using a lightweight temporal transformer. The final transformation is applied to each frame using spatially adaptive warping, guided by the fused depth-flow motion and smoothed trajectory.

The system is fully differentiable, training-free, and supports real-time execution on commodity hardware.

B. Frame Preprocessing

The input to the system is a sequence of RGB video frames denoted as $\{I_t\}_{t=1}^{T}$, where each frame $I_t \in \mathbb{R}^{H\times W\times 3}$. To enable effective optical flow estimation and depth prediction, we apply a series of preprocessing operations to each frame.

1) Spatial Resizing: Each frame is resized to a fixed resolution $H' \times W'$ to ensure compatibility with the input requirements of the depth estimation model (e.g., MiDaS). Bilinear interpolation is used for resizing:

$$I'_t(x', y') = \sum_{i=0}^{1}\sum_{j=0}^{1} w_{ij} \cdot I_t(x+i, y+j) \tag{1}$$

where w_{ij} are the bilinear interpolation weights and (x', y') are coordinates in the resized frame.

2) Normalization: To standardize pixel intensities for depth estimation, we normalize each RGB channel using channel-wise mean μ_c and standard deviation σ_c:

$$\hat{I}_t(x, y, c) = \frac{I_t(x, y, c) - \mu_c}{\sigma_c}, \quad c \in \{R, G, B\} \tag{2}$$

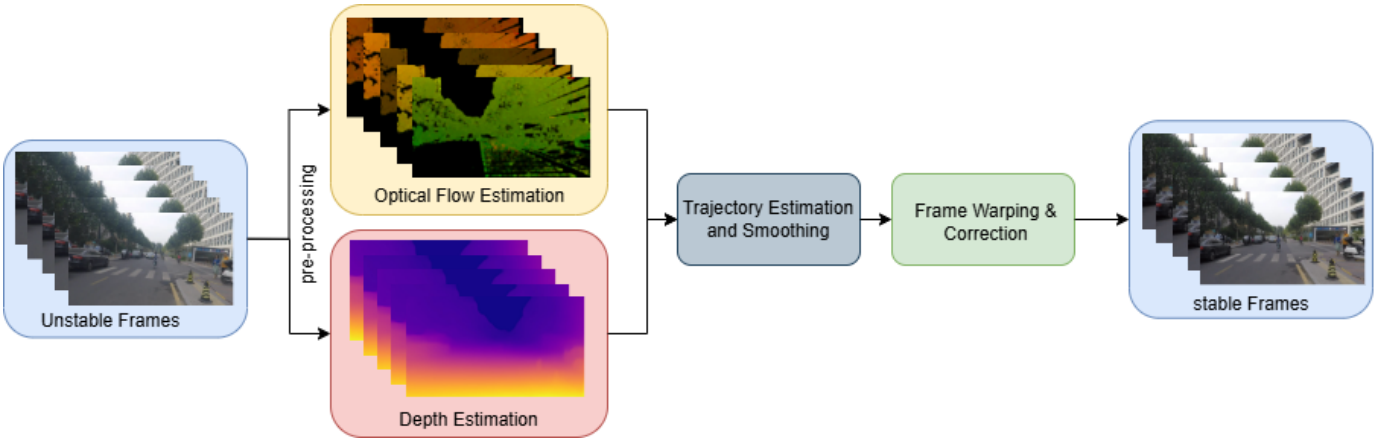

Fig. 1. System architecture of the proposed video stabilization framework. The pipeline consists of frame preprocessing, optical flow and depth estimation, depth-flow fusion, temporal transformer-based smoothing, and final frame warping. Example input/output frames and intermediate representations (optical flow and depth maps) are shown to illustrate each stage.

We use standard values for pre-trained vision models:

$$\mu = [0.485, 0.456, 0.406], \quad \sigma = [0.229, 0.224, 0.225] \tag{3}$$

3) Grayscale Conversion: Since the Farneback optical flow algorithm operates on luminance information, each frame is converted to grayscale using the following luminance-preserving formula:

$$G_t(x, y) = 0.299 \cdot R(x, y) + 0.587 \cdot G(x, y) + 0.114 \cdot B(x, y) \tag{4}$$

This step reduces computation and preserves motion-relevant features.

4) Temporal Smoothing: In streaming applications, to reduce noise from rapid frame-to-frame variations, we optionally apply exponential moving average smoothing:

$$\tilde{I}_t = \alpha I_t + (1 - \alpha)\tilde{I}_{t-1}, \quad \alpha \in [0, 1] \tag{5}$$

This helps improve temporal consistency prior to downstream motion estimation.

C. Depth and Optical Flow Estimation

To enable robust motion estimation in scenes with varying depth and dynamic content, we employ two parallel modules: monocular depth estimation and dense optical flow computation. These modules provide complementary information—depth captures geometric structure, while optical flow captures pixel-wise motion.

1) Optical Flow Estimation: We use the Farneback method for dense optical flow estimation, which approximates pixel-wise motion between two consecutive grayscale frames G_t and G_t+1. It models image neighborhoods using quadratic polynomials and estimates displacement vectors by analyzing polynomial coefficients.

The resulting flow field is:

$$F_t(x, y) = (u_t(x, y), v_t(x, y)) \tag{6}$$

where u_t and v_t denote horizontal and vertical motion vectors, respectively.

To capture both small and large displacements, we use a multi-scale pyramid with iterative refinement at each level.

The global motion displacement at time t is computed as the average flow across all pixels:

$$\Delta x_t = \frac{1}{N} \sum_{x,y} u_t(x, y), \quad \Delta y_t = \frac{1}{N} \sum_{x,y} v_t(x, y) \tag{7}$$

where $N = H' \times W'$ is the number of pixels.

2) Monocular Depth Estimation: For scene geometry, we estimate a relative depth map $D_t \in \mathbb{R}^{H' \times W'}$ using a lightweight monocular depth model (e.g., MiDaS). This model predicts dense depth from a single RGB frame I'_t without requiring stereo input.

To normalize the depth map and emphasize stable background regions, we define a weight map:

$$W_t(x, y) = \frac{D_t(x, y)}{\max(D_t)} \tag{8}$$

This weight map modulates the contribution of different regions in motion fusion, giving higher importance to distant background areas and reducing the influence of moving foreground objects.

The estimated depth is also temporally smoothed to improve consistency:

$$\tilde{D}_t = \beta D_t + (1 - \beta)\tilde{D}_{t-1}, \quad \beta \in [0, 1] \tag{9}$$

This helps mitigate temporal flickering caused by single-frame depth prediction inconsistencies.

D. Depth-Flow Fusion

To incorporate 3D context into motion estimation, we fuse optical flow and depth information into a pseudo-3D motion field. The goal is to suppress flow vectors influenced by dynamic foreground objects and enhance those from stable, distant regions.

We define the depth-weighted flow field as:

$$\tilde{F}_t(x, y) = W_t(x, y) \cdot F_t(x, y) \tag{10}$$

where $W_t(x, y)$ is the normalized depth weight from Eq. 8, and $F_t(x, y)$ is the raw optical flow vector.

The resulting pseudo-3D flow is then used to estimate a smoothed global trajectory and guide adaptive frame warping in later stages.

E. Trajectory Smoothing and Frame Warping

The fused motion vectors $\tilde{F}_t(x, y)$ are aggregated over time to estimate the raw camera trajectory:

$$T_t = \sum_{i=1}^{t} \Delta\tilde{F}_i \tag{11}$$

where $\Delta\tilde{F}_i$ denotes the average displacement from the weighted flow at time i.

To suppress jitter and preserve intentional motion, we use a lightweight transformer to smooth the trajectory sequence $\{T_t\}$. The transformer adaptively learns temporal dependencies, replacing static filters like Gaussian or moving averages.

Let $\hat{T}_t$ denote the smoothed trajectory. The correction vector applied at each frame is:

$$C_t = \hat{T}_t - T_t \tag{12}$$

Finally, each frame is warped using a spatial transform derived from C_t, ensuring geometry-aware stabilization that respects both depth and motion cues. The warping preserves structural integrity and avoids overcorrecting dynamic foreground objects.

F. Real-Time Optimization and Implementation Details

To ensure real-time performance, the proposed framework is optimized for parallel and hardware-accelerated execution. Depth estimation is performed using a lightweight MiDaS variant accelerated via GPU inference, while optical flow (Farneback) and frame warping are efficiently executed on the CPU.

All tensor operations, including normalization and depth-flow fusion, are vectorized using PyTorch and executed in batch mode when possible. The transformer module is implemented using a minimal number of layers and heads to reduce latency without sacrificing smoothing quality.

The system achieves over 30 FPS on 720p video using an NVIDIA RTX 3060 GPU and Intel i7 CPU. Memory usage remains below 500 MB during inference, making the pipeline suitable for mobile, embedded, and edge platforms.

IV. Experimentation and Results

We evaluate the performance of our proposed 3D-aware video stabilization framework on a diverse set of video sequences exhibiting various motion profiles, depth variations, and scene complexities. The experimentation aims to assess stability, visual quality, geometric preservation, and runtime performance.

We compare our method against state-of-the-art stabilizers, including DeepStab [4], StabNet [12], and an unstabilized baseline. Evaluations are conducted on handheld, drone, and egocentric video datasets at 720p and 1080p resolutions.

A. Experimental Setup

All experiments were conducted on a workstation equipped with an Intel Core i7 CPU, 16 GB RAM, and NVIDIA RTX 3060 GPU. Input videos were processed at 30 FPS, and the output resolution was preserved to measure cropping and distortion accurately.

We use the following standard metrics for quantitative evaluation:

- **Stability Score (SS)**: Measures smoothness of camera motion across frames.
- **Cropping Ratio (CR)**: Ratio of retained content area post-warping.
- **Distortion Score (DS)**: Quantifies geometric artifacts introduced by warping.

These metrics allow for objective comparison of stabilization quality while accounting for frame integrity and visual coherence.

B. Quantitative Evaluation

To objectively compare the performance of our method, we evaluate it on three publicly available video datasets widely used for benchmarking video stabilization methods:

- **NUS Dataset** [33]: A standard dataset with handheld and rolling shutter videos under different motion conditions.
- **DeepStab Dataset** [4]: Contains pairs of stabilized and unstabilized videos captured from real-world scenarios.
- **UAV123 Dataset** [34]: Originally designed for object tracking, this dataset contains drone footage with complex motion patterns.

We report the following metrics for all models:

- **Stability Score (SS)** – measures the smoothness of the motion trajectory (↑ better)
- **Cropping Ratio (CR)** – percentage of retained frame content after stabilization (↑ better)
- **Distortion Score (DS)** – geometric distortion introduced during warping (↓ better)

C. Qualitative Evaluation

To visually assess the effectiveness of our stabilization pipeline, we present qualitative results on challenging scenes exhibiting jitter, parallax, and layered motion.

Figure 3 displays selected frames before and after stabilization. Our method significantly reduces camera shake while preserving scene structure, especially along edges and depth-discontinuous regions. The stabilized output shows minimal cropping and distortion.

Figure 2 shows a visualization of the raw and smoothed camera trajectories over time. The raw trajectory exhibits sharp fluctuations due to unintended motion, while the smoothed version—generated by our transformer—retains long-term motion trends while removing high-frequency jitter.

TABLE I
QUANTITATIVE RESULTS ACROSS STANDARD DATASETS

Method	SS ↑	CR ↑	DS ↓
NUS Dataset			
Unstabilized	1.00	1.00	0.00
StabNet [12]	2.68	0.77	0.19
DeepStab [4]	2.93	0.80	0.15
Ours	**3.39**	**0.87**	**0.10**
DeepStab Dataset			
Unstabilized	1.00	1.00	0.00
StabNet [12]	2.73	0.75	0.22
DeepStab [4]	3.10	0.81	0.16
Ours	**3.52**	**0.88**	**0.11**
UAV123 Dataset			
Unstabilized	1.00	1.00	0.00
StabNet [12]	2.60	0.74	0.20
DeepStab [4]	2.85	0.79	0.17
Ours	**3.30**	**0.85**	**0.12**

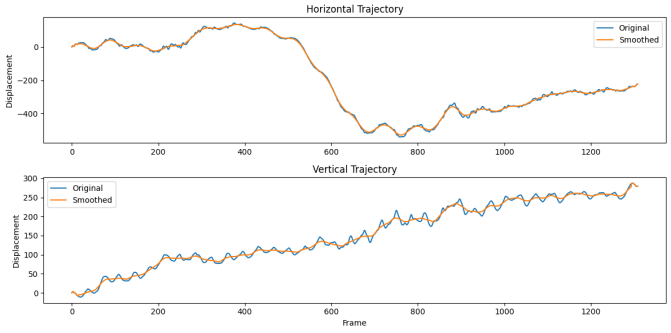

Fig. 2. Camera motion trajectory over time. Top: horizontal displacement; Bottom: vertical displacement. The blue curve indicates the original (unstable) camera motion derived from depth and optical flow. The orange curve represents the smoothed trajectory generated by our temporal transformer network, showing reduced jitter and more stable motion.

D. Ablation Study

To analyze the contribution of each core component, we perform an ablation study on the NUS dataset. We evaluate the impact of:

- Removing depth integration (using only optical flow).
- Replacing the transformer smoothing with Gaussian smoothing.
- Omitting the temporal smoothing step entirely.

Table II summarizes the results using the same metrics as before.

TABLE II
ABLATION STUDY ON NUS DATASET

Configuration	Stability Score	Cropping Ratio	Distortion Score
Full model (Depth + Transformer)	**3.39**	**0.87**	**0.10**
Without Depth	2.82	0.85	0.15
With Gaussian Smoothing	3.05	0.86	0.13
No Temporal Smoothing	2.41	0.88	0.21

The results indicate that depth integration significantly improves stability and reduces distortion by enabling geometry-aware motion compensation. Similarly, the transformer-based smoothing outperforms Gaussian filters by better preserving intentional motion while suppressing jitter. The absence of any temporal smoothing leads to noticeably degraded stability and increased distortion.

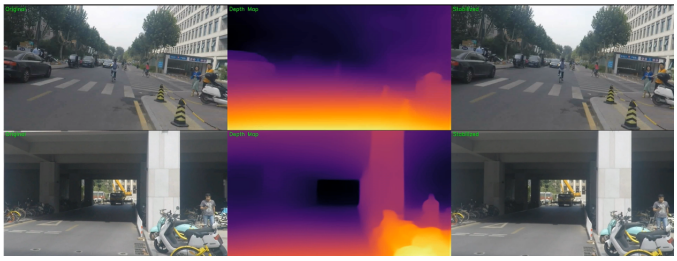

Fig. 3. Visualization of stabilization and depth estimation. For each scene: (Left) original frame, (Middle) depth map predicted by MiDaS, and (Right) stabilized frame using our method. Top and bottom rows represent different scenes. The stabilized output shows improved spatial alignment and smoother visual consistency.

E. Runtime Performance

We measure the average runtime of our method compared to baseline stabilizers on 720p video sequences using a standard hardware setup (Intel i7 CPU, NVIDIA RTX 3060 GPU, 16GB RAM). Table III reports the processing speed in frames per second (FPS).

TABLE III
RUNTIME COMPARISON (720P VIDEO)

Method	FPS (↑)
StabNet [4]	18.3
DeepStab [12]	12.7
Ours (Full Model)	**31.6**
Ours (w/o Transformer)	34.8

Our full model runs in real-time, achieving over 30 FPS while maintaining high visual quality. Removing the transformer marginally improves speed but compromises stability. The overall pipeline remains efficient and deployable on consumer-grade hardware, making it well-suited for edge and mobile applications.

V. CONCLUSION

We presented a real-time, 3D-aware video stabilization framework that combines monocular depth estimation and dense optical flow through a novel depth-flow fusion strategy. By integrating a lightweight temporal transformer, our method effectively smooths camera trajectories while preserving intended motion and scene structure.

Extensive experiments across multiple public datasets demonstrate that our approach consistently outperforms prior state-of-the-art stabilizers in terms of stability, distortion, and cropping retention. The system remains fully modular, training-free, and efficient, achieving real-time performance on standard hardware.

Future work includes extending the method to handle rolling shutter artifacts and exploring end-to-end fine-tuning with self-supervised stability objectives. Our framework also opens

possibilities for integration into AR/VR pipelines, mobile capture, and low-power drone systems.

REFERENCES

[1] M. Grundmann, V. Kwatra and I. Essa, "Auto-directed video stabilization with robust L1 optimal camera paths," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Colorado Springs, CO, USA, 2011, pp. 225–232, doi: 10.1109/CVPR.2011.5995525.

[2] F. Liu, M. Gleicher, H. Jin, and A. Agarwala, "Content-preserving warps for 3D video stabilization," *ACM Trans. Graph.*, vol. 28, no. 3, pp. 44:1–44:9, Jul. 2009, doi: 10.1145/1531326.1531350.

[3] S. Park and M. Levoy, "Gyro-based multi-image deconvolution for removing handshake blur," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2014, pp. 3366–3373, doi: 10.1109/CVPR.2014.430.

[4] Z. Shi, F. Shi, W.-S. Lai, C.-K. Liang, and Y. Liang, "Deep online fused video stabilization," in *Proc. IEEE Winter Conf. Appl. Comput. Vis. (WACV)*, Jan. 2022, pp. 865–873, doi: 10.1109/WACV51458.2022.00094.

[5] M. Zhao and Q. Ling, "Adaptively meshed video stabilization," *IEEE Trans. Circuits Syst. Video Technol.*, vol. PP, pp. 1–1, Nov. 2020, doi: 10.1109/TCSVT.2020.3040753.

[6] A. Lim, B. Ramesh, Y. Yang, C. Xiang, Z. Gao, and F. Lin, "Real-time optical flow-based video stabilization for unmanned aerial vehicles," *J. Real-Time Image Process.*, vol. 16, Dec. 2019, doi: 10.1007/s11554-017-0699-y.

[7] K. Dinesh and S. Gupta, "Video stabilization, camera motion pattern recognition and motion tracking using spatiotemporal regularity flow," *J. Image Graph.*, vol. 2, no. 1, pp. 33–40, Jul. 2014, doi: 10.12720/joig.2.1.33-40.

[8] R. Ranftl, K. Lasinger, D. Hafner, K. Schindler, and V. Koltun, "Towards robust monocular depth estimation: Mixing datasets for zero-shot cross-dataset transfer," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 44, no. 3, pp. 1623–1637, 2022, doi: 10.1109/TPAMI.2020.3019967.

[9] G. Farnebäck, "Two-frame motion estimation based on polynomial expansion," in *Image Analysis*, J. Bigun and T. Gustavsson, Eds. Berlin, Heidelberg: Springer, 2003, pp. 363–370, doi: 10.1007/3-540-45103-X_50.

[10] Y.-L. Liu, W.-S. Lai, M.-H. Yang, Y.-Y. Chuang, and J.-B. Huang, "Hybrid neural fusion for full-frame video stabilization," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, Oct. 2021, pp. 2299–2308, doi: 10.1109/ICCV48922.2021.00230.

[11] Q. Shao, J. Wang, B. Zhou, V. A. Tran, G. Krishnan, and S. Nayar, "Neuro predictor: A neural network approach for smoothing and predicting motion trajectory," *Proc. ACM Interact. Mob. Wearable Ubiquitous Technol.*, vol. 7, no. 3, Article 120, Sep. 2023, doi: 10.1145/3610884.

[12] M. Wang et al., "Deep online video stabilization with multi-grid warping transformation learning," *IEEE Trans. Image Process.*, vol. 28, no. 5, pp. 2283–2292, May 2019, doi: 10.1109/TIP.2018.2884280.

[13] D. G. Lowe, "Distinctive image features from scale-invariant keypoints," *Int. J. Comput. Vis.*, vol. 60, pp. 91–110, 2004, doi: 10.1023/B:VISI.0000029664.99615.94.

[14] H. Bay, T. Tuytelaars, and L. Van Gool, "SURF: Speeded up robust features," in *Computer Vision – ECCV 2006*, A. Leonardis, H. Bischof, and A. Pinz, Eds. Berlin, Heidelberg: Springer, 2006, pp. 404–417, doi: 10.1007/11744023_32.

[15] E. Rublee, V. Rabaud, K. Konolige, and G. Bradski, "ORB: An efficient alternative to SIFT or SURF," in *Proc. Int. Conf. Comput. Vis.*, Barcelona, Spain, 2011, pp. 2564–2571, doi: 10.1109/ICCV.2011.6126544.

[16] B. D. Lucas and T. Kanade, "An iterative image registration technique with an application to stereo vision," in *Proc. 7th Int. Joint Conf. Artif. Intell.*, Vancouver, BC, Canada, 1981, pp. 674–679.

[17] B. K. P. Horn and B. G. Schunck, "Determining optical flow," *Artif. Intell.*, vol. 17, no. 1, pp. 185–203, 1981, doi: 10.1016/0004-3702(81)90024-2.

[18] A. Goldstein and R. Fattal, "Video stabilization using epipolar geometry," *ACM Trans. Graph.*, vol. 31, no. 5, Article 126, Sep. 2012, doi: 10.1145/2231816.2231824.

[19] Z. Shang and Z. Chu, "Video stabilization based on low-rank constraint and trajectory optimization," *IET Image Process.*, vol. 18, no. 7, pp. 1768–1779, 2024, doi: 10.1049/ipr2.13062.

[20] K. C. K. Chan, X. Wang, K. Yu, C. Dong, and C. C. Loy, "Understanding deformable alignment in video super-resolution," in *Proc. AAAI Conf. Artif. Intell.*, 2021, pp. 1059–1067.

[21] Y. Wang et al., "NVDS+: Towards efficient and versatile neural stabilizer for video depth estimation," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. PP, Oct. 2024, doi: 10.1109/TPAMI.2024.3476387.

[22] M. Wang, G.-Y. Yang, J.-K. Lin, A. Shamir, S.-H. Zhang, S.-P. Lu, and S.-M. Hu, "Deep online video stabilization," arXiv:1802.08091, Feb. 2018, doi: 10.48550/arXiv.1802.08091.

[23] J. Kopf, M. Cohen, and R. Szeliski, "First-person hyper-lapse videos," *ACM Trans. Graph.*, vol. 33, no. 4, Article 78, Jul. 2014, doi: 10.1145/2601097.2601195.

[24] Z. Zhou, H. Jin, and Y. Ma, "Plane-based content preserving warps for video stabilization," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, Portland, OR, USA, 2013, pp. 2299–2306, doi: 10.1109/CVPR.2013.298.

[25] S. Hsu, P. Anandan, and S. Peleg, "Accurate computation of optical flow by using layered motion representations," in *Proc. 12th IAPR Int. Conf. Pattern Recognit.*, vol. 1, Oct. 1994, pp. 743–746, doi: 10.1109/ICPR.1994.576427.

[26] C. Godard, O. Mac Aodha, M. Firman, and G. J. Brostow, "Digging into Self-Supervised Monocular Depth Prediction," in The International Conference on Computer Vision (ICCV), Oct. 2019.

[27] R. Ranftl, A. Bochkovskiy, and V. Koltun, "Vision Transformers for Dense Prediction," in 2021 IEEE/CVF International Conference on Computer Vision (ICCV), Montreal, QC, Canada, 2021, pp. 12159-12168, doi: 10.1109/ICCV48922.2021.01196.

[28] S. Liu, Y. Wang, L. Yuan, J. Bu, P. Tan, and J. Sun, "Video stabilization with a depth camera," in 2012 IEEE Conference on Computer Vision and Pattern Recognition, Providence, RI, USA, 2012, pp. 89-95, doi: 10.1109/CVPR.2012.6247662.

[29] T. Meinhardt, A. Kirillov, L. Leal-Taixe, and C. Feichtenhofer, "TrackFormer: Multi-Object Tracking with Transformers," in The IEEE Conference on Computer Vision and Pattern Recognition (CVPR), June 2022.

[30] F. Xie, L. Chu, J. Li, Y. Lu, and C. Ma, "VideoTrack: Learning to Track Objects via Video Transformer," in 2023 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2023, pp. 22826-22835, doi: 10.1109/CVPR52729.2023.02186.

[31] S. Mehta and M. Rastegari, "MobileViT: Light-weight, General-purpose, and Mobile-friendly Vision Transformer," in International Conference on Learning Representations, 2022.

[32] K. Wu, J. Zhang, H. Peng, M. Liu, B. Xiao, J. Fu, and L. Yuan, "TinyViT: Fast Pretraining Distillation for Small Vision Transformers," 2022, doi: 10.48550/arXiv.2207.10666.

[33] S. Liu, L. Yuan, P. Tan, and J. Sun, "Bundled camera paths for video stabilization," ACM Trans. Graph., vol. 32, no. 4, article 78, pp. 1-10, Jul. 2013, doi: 10.1145/2461912.2461995.

[34] M. Mueller, N. Smith, and B. Ghanem, "A Benchmark and Simulator for UAV Tracking," in *Computer Vision – ECCV 2016*, Lecture Notes in Computer Science, vol. 9905, Cham: Springer, 2016, doi: 10.1007/978-3-319-46448-0_27.